



ITI · AI AGENTS DEVELOPMENT USING PYTHON TRACK

Python Framework with FastAPI

Eng. Fatma Elraey

Lecture 5 — Test & Deliver the Simplified ORM API

Field	Detail
Instructor	Eng. Fatma Elraey
Course	Python Framework with FastAPI
Session	Lecture 5 of 5
Focus	Test the Lecture 4 SQLite + SQLAlchemy ORM API with Swagger, curl and Postman; verify status codes, JSON contracts, migrations and the direct Session(engine) CRUD flow.

Welcome!

This session starts from the completed Lecture 4 project: the same documents.db file, the description migration, APIRouter routes, Pydantic schemas, and the direct with Session(engine) as db pattern. We now test the API from a client point of view and prepare a clean final delivery.

Today's Outline

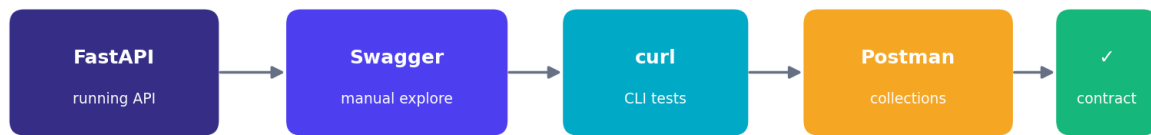
1. Verify the Lecture 4 project and HTTP contract
2. Test the API from Swagger UI
3. Test the same endpoints with curl
4. Build a reusable Postman collection
5. Final integration and delivery checklist

01

From Lecture 4 to HTTP Contract Testing

Before testing, confirm that the Lecture 4 schema is current and the API starts normally. A route is correct only when an HTTP client sees the expected method, URL, request body, status code, JSON response and failure behavior. Testing a Python function alone does not verify the full API boundary.

Test the HTTP contract from multiple client perspectives



Status code + JSON body + headers + error behavior = API contract

Figure 5.1 — Swagger, curl, and Postman exercise the same public HTTP contract from different client perspectives.

- **Happy path** — a valid request returns the expected success status and response model.
- **Validation path** — invalid Pydantic input returns 422 before the endpoint logic runs.
- **Missing resource** — an unknown document id returns 404, not 500.
- **Migration/startup path** — the database is on the expected Alembic revision before normal API testing.

✓ QUICK CHECK

Q1. What should an API test verify besides JSON content?

Answer: The HTTP status code and the rest of the public contract.

Q2. Why check Alembic before final testing?

Answer: The ORM model and documents.db schema must be on the expected version.

02

Test the Lecture 4 API with curl

curl exercises the same public API without a GUI. The examples below use Windows CMD syntax to match the course demo environment.

```
# Check migration first
python -m alembic current

# List documents
curl -i http://127.0.0.1:8000/documents

# Create (Windows CMD)
curl -i -X POST http://127.0.0.1:8000/documents ^
-H "Content-Type: application/json" ^
-d "{\"title\":\"Curl Doc\",\"content\":\"Lecture 5 test\",\"priority\":2,\"description\":\"after migration\"}"

# Read one
curl -i http://127.0.0.1:8000/documents/1

# Delete one
curl -i -X DELETE http://127.0.0.1:8000/documents/1
```

The `-i` flag shows the status line and response headers. For POST/PUT JSON requests, send `Content-Type: application/json` and a serialized JSON body. Record both the status code and the response body.

03 Postman Collections & Environments

Postman groups requests into a collection and stores reusable variables such as `base_url` and `document_id`. Build the collection around the exact Lecture 4 lifecycle and include both successful and failing requests.

Request	Method / Path	Expected
List documents	GET /documents	200
Create valid	POST /documents	201
Create invalid	POST /documents	422
Read existing	GET /documents/{id}	200
Read missing	GET /documents/999999	404
Update existing	PUT /documents/{id}	200
Delete existing	DELETE /documents/{id}	200

```
# Instructor example — Postman variables
base_url    http://127.0.0.1:8000
document_id set from the create response or manually
REQUEST     {{base_url}}/documents/{{document_id}}
```

04 Swagger UI as an Interactive Contract Explorer

Swagger UI is the fastest way to inspect the generated Lecture 4 contract and try requests from `/docs`. Compare the documented Pydantic schemas and response codes with the actual behavior you also observe from `curl` and Postman.

- **Try it out** — create, read, update and delete a document directly from `/docs`.
- **Schemas** — confirm `DocumentCreate` and `DocumentResponse` include the optional description field.
- **Responses** — verify 200/201/404/422 documentation matches what the running API actually returns.

05

Final Integration Checklist

Layer	Ready When...
Startup	python -m alembic current is correct and uvicorn main:app --reload starts cleanly.
Routing	GET/POST/PUT/DELETE routes are grouped with APIRouter and main.py includes the router.
Validation	DocumentCreate / DocumentResponse enforce and document the JSON contract.
Database	SQLite persists data using the direct Session(engine) CRUD pattern from Lectures 3–4.
Migrations	Alembic records and applies the description schema change to documents.db.
Docs	/docs, /redoc and /openapi.json describe the actual routes and response models.
Errors	Invalid input returns 422 and missing documents return 404.
Testing	Swagger, curl and Postman cover success, validation, missing-resource and mutation cases.

The final project is ready when the Lecture 4 architecture, database revision, API documentation and client-visible behavior all agree. In this simplified course version, database endpoints keep the direct with Session(engine) as db pattern.

✓ QUICK CHECK

Q1. What is the final deliverable?

Answer: A documented, migrated, persistent multi-endpoint REST API that HTTP clients can test.

Q2. Which session pattern do we keep in this simplified course?

Answer: with Session(engine) as db.

Work through these on your own before checking the marked answers.

Practice Question 1

What does `curl -i` add to the output?

- A. Response headers/status line
- B. Database tables
- C. HTML templates
- D. Alembic migration

Practice Question 2

What should a Postman collection contain?

- A. Only happy-path GET requests
- B. Success and failure cases across the API lifecycle
- C. Only screenshots
- D. Only database SQL

Practice Question 3

What is the expected status for an unknown resource id?

- A. 201
- B. 404
- C. 422 always
- D. 301

Practice Question 4

What is the purpose of an environment variable such as `base_url` in Postman?

- A. Avoid repeating environment-specific values
- B. Create ORM models
- C. Render HTML
- D. Disable HTTP

Practice Question 5

Which statement best describes the completed Lecture 4/5 API?

- A. It only starts without errors
- B. It has explicit contracts, SQLite persistence, Alembic migrations, APIRouter organization, docs, and tests
- C. It must render server-side HTML
- D. It requires one endpoint only

Answers

Practice Question 1: A ✓ — It includes response headers, including the HTTP status line.

Practice Question 2: B ✓ — A useful collection verifies the full public contract.

Practice Question 3: B ✓ — The specific resource does not exist.

Practice Question 4: A ✓ — Variables make collections reusable across environments.

Practice Question 5: B ✓ — Readiness comes from the combined behavior of the Lecture 4 architecture and the Lecture 5 tests.